

Data Analytics Basic Methods

Assignment 1 (10%)

[Zion Yitbarek]

[Insert course section & student number]

This assignment can be submitted using either Python or R, whichever you prefer. If using R, you must submit an RMD file with its knitted file (PDF or HTML). To learn more about knitting and R markdown, visit R Markdown.

If using Python, you must submit an IPYNB file and its exported PDF/HTML with clearly printed/shown answers.

Failing to submit both files ({RMD + knitted PDF/HTML} OR {IPYNB + PDF/HTML}) will be subject to a 30% mark deduction.

NOTE: IF YOU USE R STUDIO , YOU SHOULD NEVER HAVE `install.packages` IN YOUR CODE; OTHERWISE, THE Knit OPTION WILL RAISE AN ERROR. COMMENT OUT ALL PACKAGE INSTALLATIONS BUT KEEP `library()` CALLS.

NOTE: If you answer the questions in R, all your answers should be in R (ignore Python questions). If you answer the questions in Python, all your answers

Question 1 (40 Points)

Q. 1a (5 points)

Create a vector of all even numbers from 3 to 49. Hint: Use `seq()` in R or `range()` in python

```
seq(4,48,2)
```

```
## [1] 4 6 8 10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48
```

Q. 1b (5 points)

Create and print a vector `x` with all integers from 1 to 80, and a vector `y` with odd integer numbers in the same range. Hint: use `seq()` function in R or `range()` in python. Calculate the difference in lengths of the vectors `x` and `y`. Hint: use `length()` in R or `len()` in python

```
x <-seq(1,80,1)
y <-seq(1,79,2)
print(x)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
## [26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
## [51] 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
## [76] 76 77 78 79 80
```

```
print (y)
```

```
## [1] 1 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49
## [26] 51 53 55 57 59 61 63 65 67 69 71 73 75 77 79
```

Q. 1c (10 points)

Create a new vector, “x_square”, with the square of elements at indices 3 5 7 9 11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 from the variable “x”. (Hint: Use indexing rather than a for loop.)

Calculate the mean and median of the LAST 7 values from x_square.

```
indices <- c(3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,41)
x_square <- x[indices]^2
print(x_square)
```

```
## [1] 9 25 49 81 121 169 225 289 361 441 529 625 729 841 961
## [16] 1089 1225 1369 1521 1681
```

```
x_square_new <- tail(x_square,7)
mean_x_square <- mean(x_square_new)
median_x_square <- median(x_square_new)
print(paste("The mean is" , mean_x_square))
```

```
## [1] "The mean is 1241"
```

```
print(paste("The median is" , median_x_square))
```

```
## [1] "The median is 1225"
```

Q. 1d (10 Points)

For a given factor variable `factorVar <- factor(c("3.5", "8.1", "5.2", "9.8"))` in R or `factorVar = ["3.5", "8.1", "5.2", "9.8"]` in python Please convert it into numeric data type and show proof that it is converted to numeric.

```
factorVar <- factor(c(3.5, 8.1, 5.2, 9.8))
numvar <- as.numeric(as.character(factorVar))
str(numvar)
```

```
## num [1:4] 3.5 8.1 5.2 9.8
```

```
print(numvar)
```

```
## [1] 3.5 8.1 5.2 9.8
```

Q. 1e (10 points)

A comma-separated values file A comma-separated values file `dataset.csv` consists of missing values represented by Not a Number (NaN) and question mark (?). How can you read this type of files in R? consists of missing values represented by Not a Number (NaN) and question mark (?). How can you read this type of files in R?

```
getwd()
```

```
## [1] "C:/Users/user/Documents"
```

```
file_1<-"C:/Users/user/Documents"  
data_1 <- read.csv(file.choose(), na.strings = c("NaN", "?"))  
print(data_1)
```

```
##  
## 1  
## 2  
## 3  
## 4  
## 5  
## 6  
## 7  
## 8  
## 9  
## 10  
## 11  
## 12  
## 13  
## 14  
## 15  
## 16  
## 17  
## 18  
## 19  
## 20  
## 21  
## 22  
## 23  
## 24  
## 25  
## 26  
## 27  
## 28  
## 29  
## 30  
## 31  
## 32  
## 33  
## 34  
## 35  
## 36  
## 37
```

38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91

```
## 92
## 93
## 94
## 95
## 96
## 97
## 98
## 99
## 100
## 101
## 102
## 103
## 104
## 105
## 106
## 107
## 108 A comma-separated values file A comma-separated values file 'dataset.csv' consists of missing va
## 109
## 110
## 111
## 112
## 113
## 114
## 115
## 116
## 117
## 118
## 119
## 120
## 121
## 122
## 123
## 124
## 125
## 126
## 127
## 128
## 129
## 130
## 131
## 132
## 133
## 134
## 135
## 136
## 137
## 138
## 139
## 140
## 141
## 142
## 143
## 144
## 145
```

146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199

```
## 200
## 201
## 202
## 203
## 204
## 205
## 206
## 207
## 208
## 209
## 210
## 211
## 212
## 213
## 214
## 215
## 216
## 217
## 218
## 219
## 220
## 221
## 222
## 223
## 224
## 225
## 226
## 227
## 228
## 229
## 230
## 231
## 232
## 233
## 234
## 235
## 236
## 237
```

Question 2 (60 Points)

Compute with using R or Python commands

Q. 2a (10 points)

$$\sum_{n=1}^{100} \frac{4^n}{(n+1)!}$$

Hint: Use `factorial(n)` to compute $n!$

```
numseq <- seq(1, 100, 1)
numseq2 <- 4^numseq
```

```
fact_seq <- factorial(numseq +1)
subtotal <- numseq2/ fact_seq
final_answer <- sum(subtotal)
print(final_answer)
```

```
## [1] 12.39954
```

Q. 2b (10 points)

$$\sum_{n=1}^{25} \left(\frac{3^n}{n^3} + \frac{n^4}{4^n} \right)$$

```
seq_num_1 <- seq(1, 25 ,1)
seq_num_2 <- 3^seq_num_1
seq_num_3 <- seq_num_1^3
term1 <-seq_num_2/seq_num_3
term2 <-seq_num_1^4/4^seq_num_1
final <- sum(term1 +term2)
print(final)
```

```
## [1] 87186876
```

Q. 2c (10 points)

$$\sum_{n=0}^{100} \frac{(n+1)5^n}{25^{n+1}}$$

```
seq_num_a <-seq(0,100,1)
numerator <- (seq_num_a +1) *5^seq_num_a
denominator <- 25^(seq_num_a +1)
subans <- numerator/denominator
finalans <- sum(subans)
print(finalans)
```

```
## [1] 0.0625
```

Q. 2d (10 points)

$$\prod_{n=2}^{24} \left(3n + \frac{3}{\sqrt[4]{n}} \right)$$

```
seq_num_b <- seq(2,24,1)

num1= 3*seq_num_b +3
num2 <- 3/(seq_num_b^(1/4))
num3 <- num1 +num2

finalnum <-prod(num3)
print(finalnum)
```

```
## [1] 2.90925e+36
```


Q. 2e (10 points)

$$\sum_{n=1}^9 \left(\frac{3^n}{n^3} + \frac{n^5}{4^5} + \frac{n^7}{7^n} + \frac{n^9}{9^n} \right)$$

(5 points)

```
final_seq <- seq(1,9,1)

part1 <-((3^final_seq)/(final_seq^3))
part2 <-part2 <- ((final_seq^5)/(4^5))
part3 <- ((final_seq^7)/(7^final_seq))
part4 <- ((final_seq^9)/(9^final_seq))

final_sum <- part1 + part2+ part3+ part4

final_total <- sum(final_sum)
print(final_total)
```

```
## [1] 338.3396
```

Q. 2f (10 points)

Describe the purpose of `is.logical()` , `is.character()` , `is.numeric()` , and `is.na()` functions in R.

In Python, describe the purpose of `isinstance()` , `type()` , and `pd.isna()` functions from the pandas library.

Hint: Please use `x <- c("a", FALSE, "b", NA, 2, TRUE)` in R or `x = ["a", False, "b", None, 2, True]` in Python to explain your description.

```
#Creating the vector
x<-c("a", FALSE, "b", NA, 2, TRUE)
```

```
#The purpose of is.logical() is to check if the value/s is logical, meaning that if the value/s are true or false
is_logical <- is.logical(x)
print(is_logical)
```

```
## [1] FALSE
```

```
#The purpose of is.character() is to check if the values/s is a string
is_character <- is.character(x)
print(is_character)
```

```
## [1] TRUE
```

```
#Final result will be TRUE for "a" and "b" and FALSE for everything else
```

```
# The purpose of is.numeric(): Checks if the value is numeric (number).
is_numeric <- is.numeric(x)
print(is_numeric)
```

```
## [1] FALSE
```

#Final result will be TRUE for the fifth element (2) and FALSE for the others as they aren't numbers.

#The purpose of is.na() is to check if there are any missing values (NA)

```
is_na <- is.na(x)  
print(is_na)
```

```
## [1] FALSE FALSE FALSE  TRUE FALSE FALSE
```

#Final result is will be FALSE for the first 3 elements ("a", FALSE, "b"), TRUE for the fourth element